COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 08/07/26 | **DATE SUBMITTED:** | 08/07/26 |
| **NAME:** | Villaruel, Prince Benjamin S. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## ACTIVITY NO. 5.1 Queues

### 1. PROCEDURE OUTPUT

What procedures did you perform?

**ILO A: Create queue using C++ STL**

amain.cpp

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Queue > ILO A > amain.cpp > main()

```cpp
#include<iostream>
#include<queue>

void display(std::queue<char> copyQ);
int main(){

    std::queue<char> myQ;

    myQ.push('P');
    myQ.push('B');
    myQ.push('V');
    std::cout<<"Current front is: "<<myQ.front()<<std::endl;
    std::cout<<"Current back is: "<<myQ.back()<<std::endl;
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    std::cout<<"Is the queue empty? "<<myQ.empty()<<std::endl;

    return 0;
}

void display(std::queue<char> copyQ){
    std::queue<char> temp = copyQ;
    while(!temp.empty()){
        std::cout<<" "<<temp.front()<<" ";
        temp.pop();
    }
    std::cout<<std::endl;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Queue\Supplementa
\" ; if ($?) { g++ amain.cpp -o amain } ; if ($?) { .\amain }
Current front is: P
Current back is: V
 P  B  V
 B  V
 V
Is the queue empty? 1
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Queue\ILO A>
```

**Observation**

For ILO A, I learned the concept of queues by doing, wherein FIFO means the first element added is the first one removed. Like push() being the first to come out when pop() is called.

### B.1. Linked List Implementation

bmain.cpp

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Queue > ILO B > bmain.cpp > main()

```cpp
#include<iostream>
#include"queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('P', &front, &back);
    std::cout<<"Front: "<<front->data <<" "<<"Back: "<<back->data<<std::endl;
    enqueue('B', &front, &back);
    std::cout<<"Front: "<<front->data <<" "<<"Back: "<<back->data<<std::endl;
    enqueue('V', &front, &back);
    std::cout<<"Front: "<<front->data <<" "<<"Back: "<<back->data<<std::endl;

    std::cout<<"Displaying All: ";
    displayAll(&front);


    std::cout<<"Testing the dequeue operator: \n";
    dequeue(&front, &back);
    std::cout<<"Front: "<<front->data <<" "<<"Back: "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"Front: "<<front->data <<" "<<"Back: "<<back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);

    std::cout<<"Is the queue empty? "<<isEmpty(&front, &back)<<std::endl;
    return 0;
}
```

bmain.cpp | queuell.h

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Queue > ILO B > queuell.h > enqueue<T>(T, qNode<T>**, qN

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

template<typename T>
class qNode {
    public:
        T data;
        qNode * next;
};

template <typename T>
qNode<T>* new_node (T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    qNode<T>* newNode = new_node(newData);

    if((*frontPtr)==nullptr && (*backPtr)==nullptr){
        (*backPtr) = newNode;
        (*frontPtr)= newNode;
        return;
    }
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    if((*frontPtr)== nullptr && (backPtr)== nullptr){
        std::cout<<"The queue is empty."<<std::endl;
        return;
    }
    qNode<T>* deleteTemp = nullptr;
    deleteTemp = (*frontPtr);

    if((*frontPtr)== nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }
    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;
}

//isEmpty
template <typename T>
bool isEmpty(qNode<T>** frontPtr, qNode<T>** backPtr){
    return((*frontPtr)== nullptr && (*backPtr) == nullptr);
}

//displayAll
template <typename T>
void displayAll(qNode<T>** frontPtr){
    qNode<T>* temp = (*frontPtr);
    while(temp != nullptr){
        std::cout<<" "<<temp->data<<" ";
        temp = temp->next;
    }
    std::cout<<std::endl;
}
#endif
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Queue\ILO A> cd
Front: P Back: P
Front: P Back: B
Front: P Back: V
Displaying All:  P  B  V
Testing the dequeue operator:
Front: B Back: V
Front: V Back: V
Is the queue empty? 1
```

## Observation

For ILO B.1, I noticed that implementing a queue using a linked list is far easier than using an array, since enqueue and dequeue just involve updating front and back pointers without shifting elements. This is the opposite of stacks, where array implementation is simpler since push/pop only affects one end, while a linked list would need extra pointer traversal just to reach that end.

## B.2. Array-Based Implementation

bmain.cpp | main.cpp | queuell.h

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Queue > ILO C > main.cpp > main()

```cpp
#include<iostream>
#include"queueArr.h"

int main(){
    //creating an object
    queueArr<int> q(5);

    //enqueue 10, 20, 30, 40, 50:
    q.Enqueue(1);
    std::cout<<"Front: " << q.front() <<std::endl;
    std::cout<<"Back: " << q.back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(2);
    std::cout<<"Front: " << q.front() <<std::endl;
    std::cout<<"Back: " << q.back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(3);
    q.Enqueue(4);
    std::cout<<"Front: " << q.front() <<std::endl;
    std::cout<<"Back: " << q.back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(5);
    std::cout<<"Front: " << q.front() <<std::endl;
    std::cout<<"Back: " << q.back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(6);
    std::cout<<"\nRemoved: "<<q.Dequeue()<<std::endl;
    std::cout<<"Removed: "<<q.Dequeue()<<std::endl;
    q.Clear();
    std::cout<<"Is the Queue Empty? "<<(q.Empty()?"Yes":"No")<<std::endl;
    return 0;
}
```

queueArr.h queueII.h

C: > Users > Administrator > Desktop > CPE21S1 - Villaruel > Queue > ILO C > queueArr.h > ...

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

//array based circular queue
template <typename T>
class queueArr{
        private:
        //pointer to dynamically allocate array
        T* q_array;
        //maximum number of elements a queue can hold
        size_t q_capacity;
        //current number of the elements in the queue
        size_t q_size;
        //index of the front elemet
        int q_front;
        //index of the rear element
        int q_back;
    public:
        //constructor
        queueArr(size_t capacity = 10);
        //copy constructor
        queueArr(const queueArr& other);
        //copy assignment operator
        queueArr& operator=(const queueArr& other);
        //destructor
        ~queueArr();
        //queue operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T front();
        T back();
        void Enqueue(T value);
        T Dequeue();
};
//Constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){
    //set capacity
    q_capacity = capacity;
    //allocated the array
    q_array = new T[q_capacity];
    q_front = 0;
    q_back = -1;
    q_size = 0;
}
//Copy Constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other){
    //copy
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    //allocate a seperate array
    q_array = new T[q_capacity];
    //copy each element
    for(size_t i = 0; i < q_capacity; i++){
        q_array[i] = other.q_array[i];
    }
}

```

```
//Copy Assignment Operator
template <typename T>
queueArr<T>& queueArr<T>::operator = (const queueArr& other
    //checking
    if(this == &other){
        return *this;
    }
    //release the previous array
    delete[] q_array;
    //copy
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    //allocate
    q_array = new T[q_capacity];
    //copy each element
    for(size_t i = 0; i < q_capacity; i++){
        q_array[i] = other.q_array[i];
    }
    return *this;
}

//Destructor
template<typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}

//Empty
template <typename T>
bool queueArr<T>::Empty(){
    return q_size == 0;
}

//Full
template <typename T>
bool queueArr<T>::Full(){
    return q_size == q_capacity;
}

//Size
template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

//Frotn
template <typename T>
T queueArr<T>::front(){
    //check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty.\n";
        return T();
    }
    //return
    return q_array[q_front];
}

//Back
template <typename T>
T queueArr<T>::back(){
    //Check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty"<<std::endl;
        return T();
    }
    //return
    return q_array[q_back];
}
```

```cpp
//Clear
template <typename T>
void queueArr<T>::Clear(){
    //reset the q_size, q_front, q_back;
    q_front = 0;
    q_back = -1;
    q_size = 0;
}

//Enqueue
template <typename T>
void queueArr<T>::Enqueue(T value){
    //check if the queue is full
    if(Full()){
        std::cout<<"Queue is full."<<std::endl;
        return;
    }
    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;
    //store the value to the back
    q_array[q_back] = value;
    //increment the q_size
    q_size++;
}

//Dequeue
template <typename T>
T queueArr<T>::Dequeue(){
    //check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty."<<std::endl;
        return T();
    }
    //create a temporary variable to store the current front
    T temp = q_array[q_front];
    ///move q_front circularly
    q_front = (q_front + 1) % q_capacity;
    //decrement the q_size
    q_size--;
    //reset the indexes if the queue become empty:
    if(Empty()){
        q_front = 0;
        q_back = -1;
    }
    //return temporary variable
    return temp;
}
#endif
```

## Observation

For this I had to relearn the big three: constructor, copy constructor, copy assignment operator, and destructor since this queue dynamically allocates its array. Without them, copying objects would just copy the pointer, causing two objects to point to the same memory.

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Queue\ILO B>

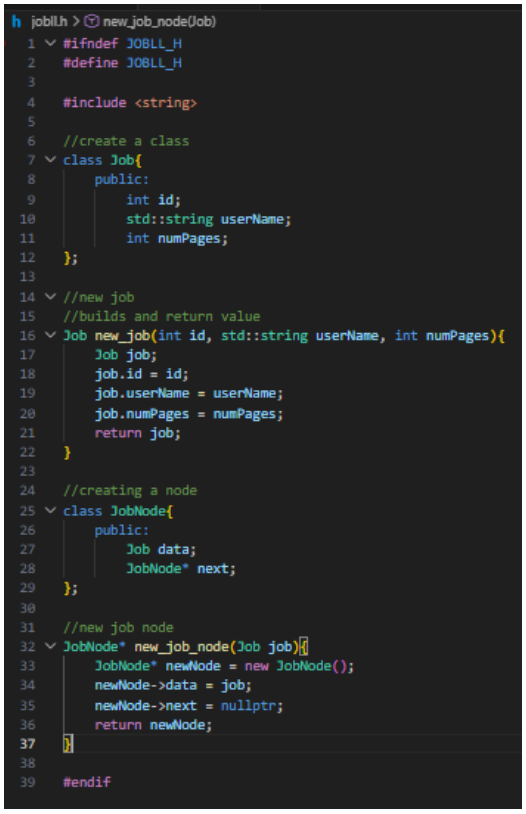
                                                    > cd "c:\Users\Administrator\Desktop\CPE21S1
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is full.

Removed: 1
Removed: 2
Is the Queue Empty? Yes
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Queue\ILO C>
```

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).



```h
jobll.h > new_job_node(Job)
#ifndef JOBLL_H
#define JOBLL_H

#include <string>

//create a class
class Job{
    public:
        int id;
        std::string userName;
        int numPages;
};

//new job
//builds and return value
Job new_job(int id, std::string userName, int numPages){
    Job job;
    job.id = id;
    job.userName = userName;
    job.numPages = numPages;
    return job;
}

//creating a node
class JobNode{
    public:
        Job data;
        JobNode* next;
};

//new job node
JobNode* new_job_node(Job job){
    JobNode* newNode = new JobNode();
    newNode->data = job;
    newNode->next = nullptr;
    return newNode;
}

#endif
```

**Observation**

- I created a job as a plain data holder with no queue logic, and new_job() to build one by value. Similarly, JobNode wraps a job with a next pointer, and new_job_node heap allocates a node and initializes next to nullptr.

Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.

To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.

```
//printer class

class Printer{
    private:
        JobNode* front;
        JobNode* back;
        int nextId;

    public:
        Printer(){
            front = nullptr;
            back = nullptr;
            nextId = 1;
        }

        void addJob(std::string userName, int numPages){
            Job job = new_job(nextId, userName, numPages);
            nextId++;

            JobNode* newNode = new_job_node(job);

            //if queue is empty, new node becomes front and back
            if(front == nullptr && back == nullptr){
                front = newNode;
                back = newNode;
                return;
            }
            //counter
            back->next = newNode;
            back = newNode;
        }

        bool isEmpty(){
            return front == nullptr;
        }

        //display all jobs pending

        void displayQueue(){
            if(isEmpty()){
                std::cout<<"No pending jobs."<<std::endl;
                return;
            }
            std::cout <<"Pending jobs: "<<std::endl;
            JobNode* current = front;
            while(current != nullptr){
                 std::cout << "  Job " << current->data.id << " - " << current->data.userName
                           << " (" << current->data.numPages << " pages)" << std::endl;
                current = current->next;
            }
        }
```

## Observation

- I correctly implemented a FIFO queue, addJob() appends to the back, and the empty queue check properly handles the first insertion by pointing both pointers at the new node. displatQueue() then walks from front to nullptr, which will print jobs in the exact order they were added.

Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.

```
    //processing

    void processOne(){
        if(isEmpty()){
            std::cout<<"No pending jobs. Printer is idle."<<std::endl;
            return;
        }
        JobNode* temp = front;
        std::cout << "Printing Job " << temp->data.id << " for " << temp->data.userName
                  << " (" << temp->data.numPages << " pages)" << std::endl;

        front = front->next;
        if(front == nullptr){
            back = nullptr;
        }
        delete temp;
    }

    //proces all jobs
    void processAll(){
        if(front == nullptr){
            std::cout << "No pending jobs."<<std::endl;
            return;
        }

        while(front != nullptr){
            JobNode* temp = front;
            std::cout << "Printing Job " << temp->data.id << " for " << temp->data.userName
                      << " (" << temp->data.numPages << " pages)" << std::endl;

            front = front->next;
            delete temp;
        }
        back = nullptr;
    }
};

#endif
```

## Observation

- For the last procedure, I managed to remove the job from the front, processOne() prints and deletes the front node, advancing front and resting back to nullptr. If the queue becomes empty, which prevents a back pointer. processAll() reuses the same logic in a loop to drain the entire queue, then resets back at once at the end.

Defend your choice of internal representation: Why did you use arrays or linked lists?

- I chose a linked list for the supplementary implementation for two main reasons. First, as I stated in ILO B.1, a linked list naturally fits a queue's concept (FIFO). Insertion at the back and removal from the front both happen at the same time, with no shifting of elements like a stack or array based approach might require. Second, a linked list can grow dynamically one node at a time instead of requiring a fixed capacity upfront which I think properly fits this activity.

suppmain.cpp

```cpp
#include <iostream>
#include "jobll.h"

int main(){
    Printer printer;

    std::cout << "Multiple users submit jobs" << std::endl;
    printer.addJob("Princee", 14);
    printer.addJob("Benjamin", 21);
    printer.addJob("Ederlyn", 7);

    std::cout << std::endl;
    printer.displayQueue();

    std::cout << std::endl << "Processing one job at a time"<<std::endl;
    printer.processOne();
    printer.processOne();

    std::cout << std::endl;
    printer.displayQueue();

    std::cout << std::endl << "More jobs arrive while printer is still working" << std::endl;
    printer.addJob("Benjie", 2);
    printer.addJob("Stefhen", 1);
    printer.addJob("Sofia", 6);
    printer.addJob("Justin", 9);

    std::cout << std::endl;
    printer.displayQueue();

    std::cout << std::endl << "Finisihing remaining queue"<<std::endl;
    printer.processOne();
    printer.processOne();
    printer.processOne();
    printer.processOne();

    return 0;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Queue\Supplementary> c
Multiple users submit jobs

Pending jobs:
  Job 1 - Princee (14 pages)
  Job 2 - Benjamin (21 pages)
  Job 3 - Ederlyn (7 pages)

Processing one job at a time
Printing Job 1 for Princee (14 pages)
Printing Job 2 for Benjamin (21 pages)

Pending jobs:
  Job 3 - Ederlyn (7 pages)

More jobs arrive while printer is still working

Pending jobs:
  Job 3 - Ederlyn (7 pages)
  Job 4 - Benjie (2 pages)
  Job 5 - Stefhen (1 pages)
  Job 6 - Sofia (6 pages)
  Job 7 - Justin (9 pages)

Finisihing remaining queue
Printing Job 3 for Ederlyn (7 pages)
Printing Job 4 for Benjie (2 pages)
Printing Job 5 for Stefhen (1 pages)
Printing Job 6 for Sofia (6 pages)
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\Queue\Supplementary>
```

**Conclusion**

- Overall, working through ILO A to ILO C gave me a much clearer picture of how queues actually work. ILO A, using the STL, showed how a simple queue can be used when the storage handles everything for you. ILO B.1 and B.2 made the trade-offs between the two implementations clearer. The array based version needed a fixed capacity and circular index math for wrap around, while linked list version grows dynamically and keeps insertion at the back and removal at the front simple, without overly complicating elements. Finally, ILO C let me apply the linked list approach to a more concrete scenario. Honestly, I'm not a huge fan of queues. It felt complicated at first, but going through all parts pushed me to slow down and really understand each step instead of relying in a built-in structure, which is a good step toward strengthening my skills.
-

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Grammarly & Google Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correction, code assistance, and explanation of concepts**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I used grammarly solely to fix grammatical errors, and I used Google Gemini strictly as a guide to help me better understand what I am doing.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.